

Report

Project by Jakob Berg & Tobias Munch

Backpropagation:

Overall approach:

The backpropagation function implements layer-wise backpropagation by iterating backwards through the network, from the output layer L to the first hidden layer. This directly follows the theoretical formulation of backpropagation, where gradients are computed using the chain rule, propagating error signals from later layers to earlier ones.

The implementation closely matches equations (4), (5), (6), and (7).

The for loop iterates backwards through each layer in the model. This reflects the definition of backpropagation, because the gradient of each layer depends on the error signal propagated from the subsequent layer.

$$\frac{\partial L}{\partial w_{i,j}^{[l]}} = \delta_i^{[l]} a_j^{[l-1]} \quad \forall l \in [1..L]$$

$$\delta_i^{[l]} = \frac{\partial L}{\partial z_i^{[l]}} = \frac{\partial L}{\partial a_i^{[l]}} \cdot \frac{\partial a_i^{[l]}}{\partial z_i^{[l]}} \quad (4)$$

$$\delta_i^{[L]} = \frac{\partial L}{\partial \hat{y}_i} \cdot \frac{\partial \hat{y}_i}{\partial z_i^{[L]}} = e'_i(\hat{y}_i) \cdot f'_i(z_i^{[L]})$$

$$e'_i(y_i, \hat{y}_i) = -2(y_i - \hat{y}_i) \quad (5)$$

$$\delta_i^{[l]} = \left(\sum_{k=1}^{n^{[l+1]}} \delta_k^{[l+1]} w_{k,i}^{[l+1]} \right) \cdot f'_i(z_i^{[l]}) \quad (6)$$

$$\frac{\partial L}{\partial b_j^{[l]}} = \delta_j^{[l]} \quad (7)$$

Output layer gradients:

we have: `delta = 2 * (y_pred - y_true) * model.df[i](model.z[i])` , which relates to the equation for the output layer gradient (5). `2*(y_pred - y_true)` comes from the derivative of the MSE loss (We use cross-entropy in the PyTorch experiments)

$$e'_i(y_i, \hat{y}_i) = -2(y_i - \hat{y}_i)$$

(the sign is absorbed in the subtraction order).

`model.df[i](model.z[i])` represents:

$$f'_i{}^{[L]}(z_i^{[L]}),$$

which is the derivative of the activation function in the input layer.

Thuss delta stores the local gradient.

The hidden layer gradients:

```
W_next = model.fc[str(i+1)].weight
```

```
delta = (delta @ W_next) * model.df[i](model.z[i]) ,
```

comes from equation (6).

`delta @ W_next` computes the weighted sums of downstream errors and `model.df[i](model.z[i])` is like before, the derivative of the activation function.

The weight gradients:

```
A_prev = model.a[i-1]
```

```
model.dL_dw[i] = (delta.t() @ A_prev)
```

This code relates to equation (4). `(delta.t() @ A_prev)` computes all the weights in one matrix operation. This avoids nested loops and follows the standard outer-product formulations. Using stored activations like `a[i-1]` avoids recomputation.

The bias gradients:

```
model.dL_db[i] = delta.sum(dim=0) ,
```

is an direct implementation of equation (7). We get the graients by summing over the deltas over the batch. This reflect the fact that the bias inputs are constant. The summation accounts for batch-wise gradient accumulation.

Conclusion:

This implementation faithfully follows the theoretical backpropagation equations (4–7). The use of vectorized operations ensures computational efficiency while preserving a clear correspondence to the mathematical definitions of local gradients, weight updates, and bias updates.

Gradient descent:

Setup

- Set seed with value 42
- Set default datatype for Pytorch as double

Data loading

The `load_cifar` function loads the CIFAR10 data while only keeping the labels airplane and bird. We use the default train and validation split 90% training data and 10% validation data. Then we are left with the following sizes of each set:

- **Train set:** 8980 images
- **Validation set:** 1020 images
- **Test set:** 2000 images

Preprocessing and analysis

All images were normalized using `torchvision.transforms.Normalize`, where the **mean** and **standard deviation** were computed from the training set.

The class distribution in the training set was approximately balanced:

- **Birds:** 49.8%
- **Planes:** 50.2%

Class labels were mapped to binary values:

- **Plane -> 0**
- **Bird -> 1**

The effect of normalization is illustrated in Figure 1.

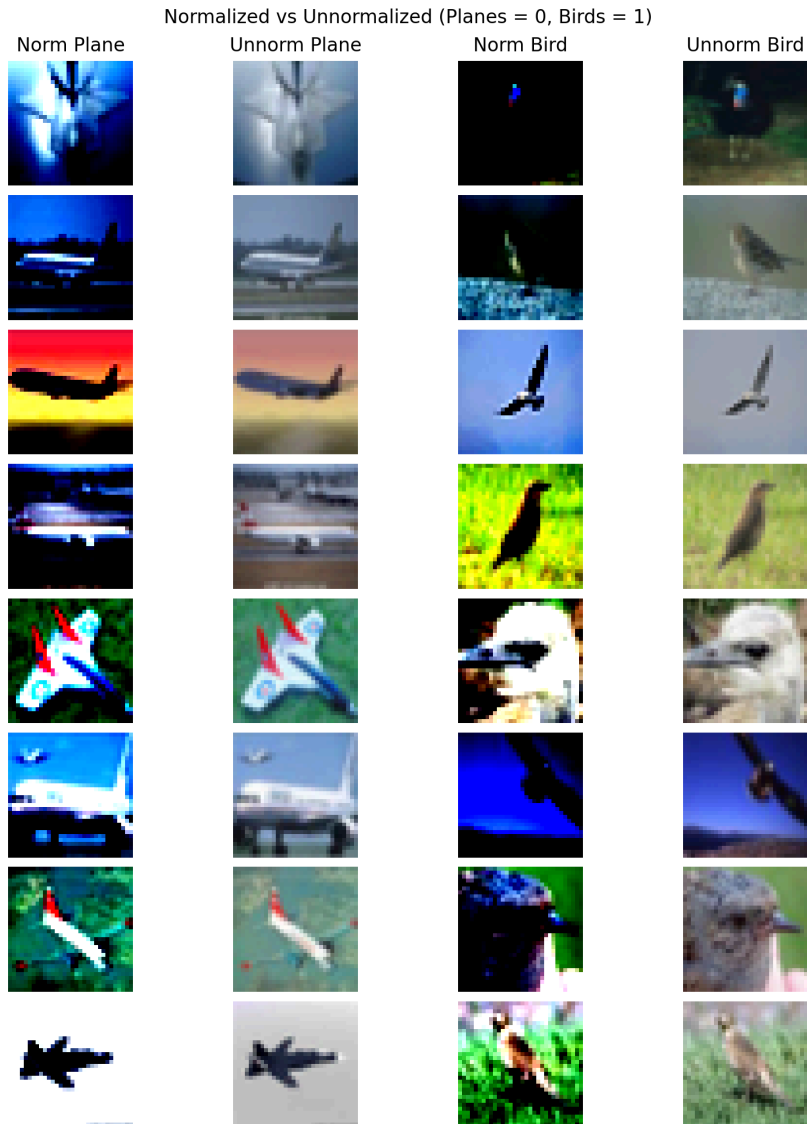


Figure 1: Effect of normalization

Model Architecture

We implement a multilayer perceptron (MLP) in PyTorch using a fully connected feedforward architecture.

The model is implemented as a configurable class with optional parameters for the hidden layer sizes and dropout rate, allowing us to compare different network architectures.

The network is implemented using `nn.Sequential`, as the data flows linearly from the input layer to the output layer without any branching or skip connections.

Each input image is first flattened from shape $3 \times 32 \times 32$ to a vector of length 3072 before being passed through the network.

The hidden layers follow a repeating **Linear -> ReLU** pattern, while the output layer consists of a linear transformation with two output units corresponding to the two classes. No activation function is applied to the output layer, as the cross-entropy loss internally applies a softmax operation.

The default architecture is:

- **Flatten**
- **Linear(3072 -> 512) → ReLU**
- **Linear(512 -> 128) → ReLU**
- **Linear(128 -> 32) → ReLU**
- **Linear(32 -> 2)**

Training

Both training functions follow the same procedure for fitting a model using gradient descent.

For a given amount of epochs we forward pass through the network to calculate the loss, compute the gradient using backpropagation, and finally update the parameters with the learning rate.

The two functions differ in how they update the parameters. The **train** function uses Pytorch's SGD optimizer meanwhile **train_manual_update** is a manual implementation of the SGD which also optionally takes in a momentum and a weight decay.

Comparison of train and train_manual_update

To compare the different functions we created a **comparison** function which has optional parameters for momentum and weight decay. We then create two identical instances of our MyNet network. We use the same loss function (CrossEntropyLoss), train loader and number of epochs for both training instances. We then create three different comparisons between the two methods:

- No weight decay or momentum
- Only weight decay
- Both weight decay and momentum

We compare the train loss of both functions with a tolerance of $1e-8$ due to float rounding etc. We then see that the two implementations are having the same training loss, one using Pytorch's SGD while one

is a manual implementation of the same function. For simplicity we will therefor continue using the simpler **train** function.

Model Comparisons with Different Architectures and Hyperparameters

To compare different network architectures and training configurations, we implemented the function `evaluate_model`.

The function performs a systematic grid search over predefined architectural choices and optimizer hyperparameters, and selects the best-performing model based on validation accuracy.

All models were trained using the **cross-entropy loss**, and performance was evaluated on the validation set.

The best model was determined as the model achieving the **highest validation accuracy** across all epochs.

Network Architectures

Four different network architectures were evaluated:

- **Baseline:** A shallow MLP with hidden layer sizes [512, 128, 32]
- **Depth:** A deeper MLP with hidden layer sizes [512, 256, 128, 64, 32]
- **Width:** A wider MLP with a larger first hidden layer [1024, 128, 32]
- **Dropout:** The baseline architecture augmented with dropout regularization (`dropout_rate = 0.3`)

These architectures allow us to investigate the effect of network depth, width, and regularization on model performance.

Optimization Hyperparameters

For each architecture, the following optimizer configurations were evaluated using stochastic gradient descent (SGD):

- Learning rate: 0.01
- Weight decay: {0.0, 1e-4, 0.01}
- Momentum: {0.0, 0.9}

This results in a total of sixteen model configurations.

All models were trained for **20 epochs** using a batch size of **64**, with a fixed random seed to ensure reproducibility.

Model Selection Criterion

For each configuration, both training and validation losses were recorded across epochs.

The validation accuracy was used as the primary performance metric, and the **maximum validation accuracy** achieved during training was used to rank models.

The configuration with the highest validation accuracy was selected as the best-performing model and retained for further evaluation on the test set.

Model comparison

We created a function `plot_best_per_arch` which plots the validation loss for the best parameter for each of the different network architectures.

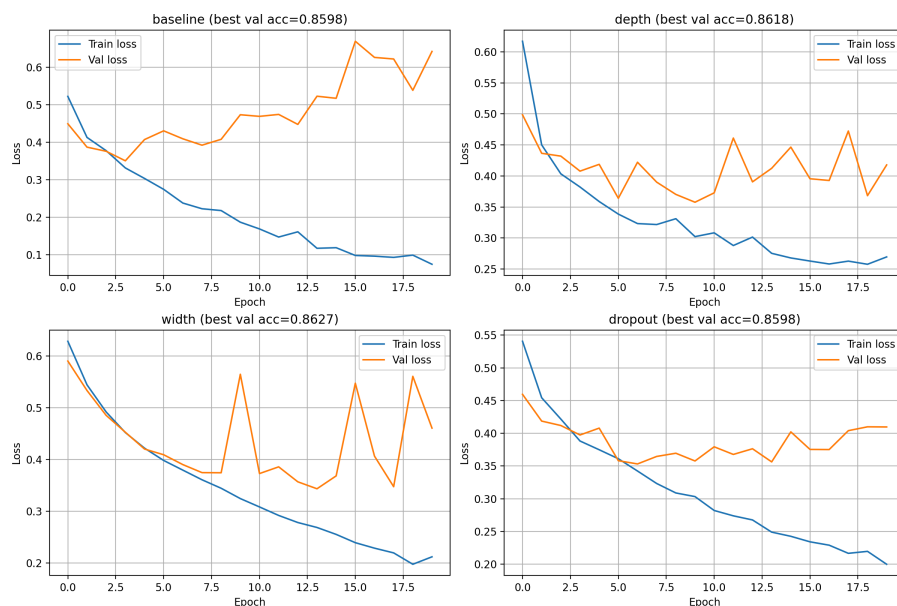


Figure 2: Validation and training loss of the four models with the highest validation accuracy

We observe that for all architectures the training loss decreases monotonically over the training period. The validation loss initially decreases, but after a certain number of epochs it starts to increase again, and some of the plots show heavy spikes.

This behavior indicates that the models begin to overfit the training data as training progresses.

Although the loss curves are used to analyze the training dynamics and overfitting behavior of the models, the final model selection is performed based on **validation accuracy**, as specified in the task description.

The validation loss and validation accuracy capture different aspects of model performance. While the loss reflects both prediction correctness and confidence, accuracy only measures whether predictions

are correct or incorrect. As a result, the model that achieves the highest validation accuracy does not necessarily correspond to the epoch with the lowest validation loss.

Consequently, the model with the highest validation accuracy is selected as the best-performing model, even though the loss curves are used for qualitative analysis of the training process.

The model with the highest accuracy was the **width** model which is built as follows:

```
Best model configuration:
Architecture name: width
Hidden sizes: (1024, 128, 32)
Learning rate: 0.01
Weight decay: 0.0
Momentum: 0.0
Epochs: 20
Dropout: 0.0
Best epoch: 18
Best validation accuracy: 0.863
```

Evaluation best model on test-data

After selecting the best model based on validation accuracy, its performance is evaluated on the test set.

The model achieves a test accuracy of **0.863**, indicating good generalization to previously unseen data.

To further analyze the classification performance, a confusion matrix is computed for the test set and shown in Figure 3.

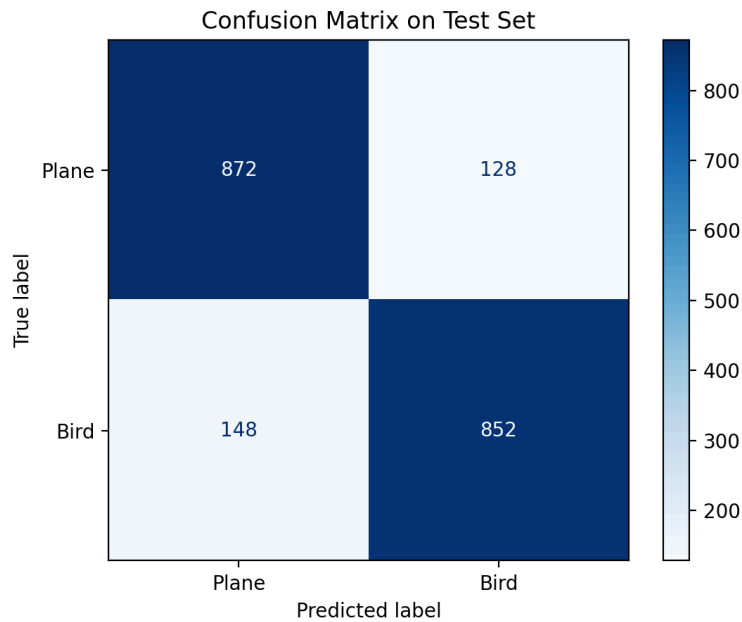


Figure 3: Confusion matrix from the test set predictions

The confusion matrix shows that the model correctly classifies **1724 out of 2000** test images. Misclassifications are relatively balanced between the two classes, with **128 plane images** misclassified as birds and **148 bird images** misclassified as planes. This suggests that the model does not exhibit a strong bias toward either class.

To further understand the model's errors, we visualize a selection of misclassified test images in Figure 4.

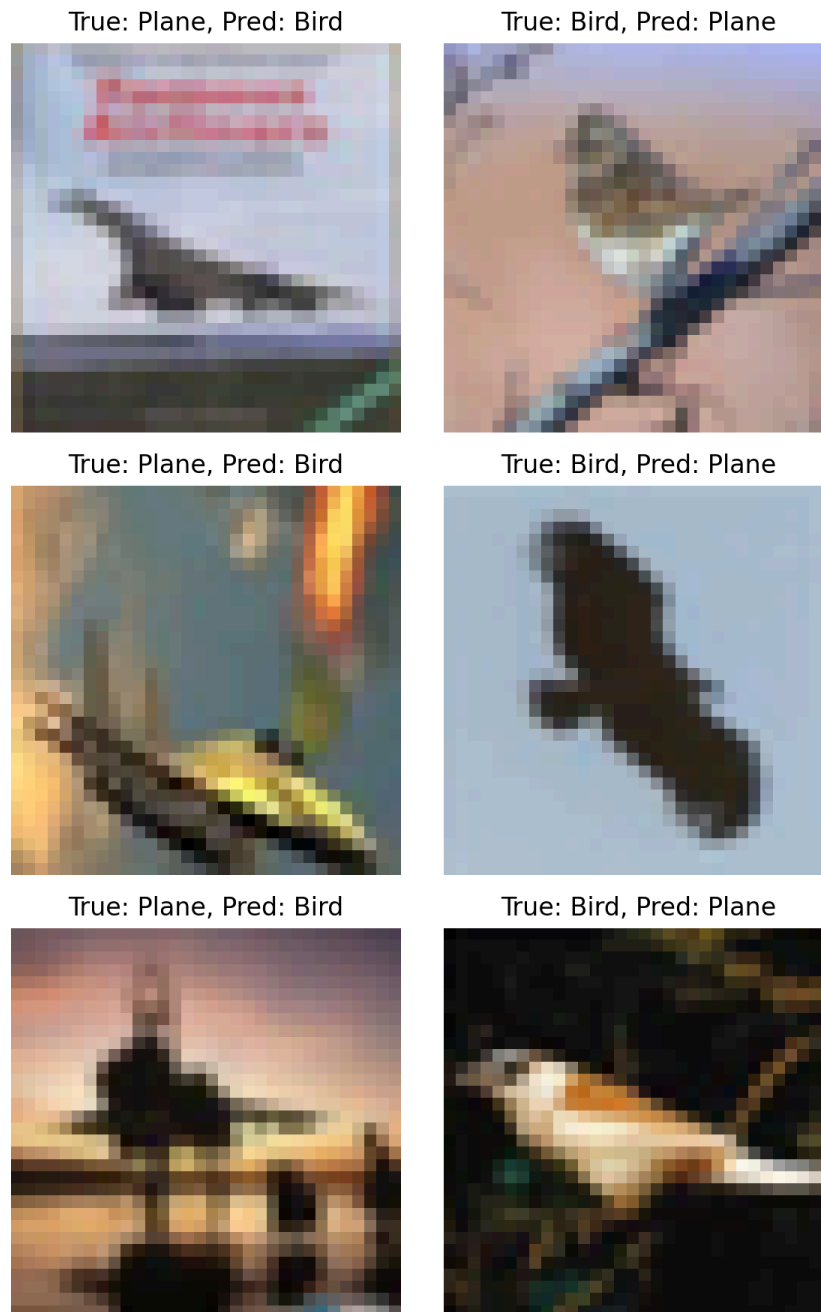


Figure 4: Visualization of some of the models misclassifications

The examples show that several misclassifications occur in images where the visual appearance of planes and birds is ambiguous. In particular, low image resolution, motion blur, and complex backgrounds make it difficult for the model to distinguish between the two classes. This suggests that the errors are primarily caused by challenging visual conditions rather than a systematic bias in the model.

Discussion of results

The experiments show that a multilayer perceptron trained with gradient descent achieves a reasonable test accuracy of 0.863 on the binary CIFAR-10 classification task. However, the performance is limited by the fact that MLPs treat images as flat vectors and therefore do not exploit the spatial structure of image data. As a result, important local patterns such as edges and shapes are not explicitly modeled, making the task challenging, especially given the low resolution and cluttered backgrounds of CIFAR-10 images.

Across all architectures, the training loss decreased monotonically, while the validation loss eventually became unstable, indicating overfitting. Increasing network depth did not improve performance and in some cases worsened generalization. The width model achieved the highest validation accuracy, despite exhibiting noticeable spikes in validation loss. This highlights the difference between loss and accuracy: while the loss reflects prediction confidence, accuracy only measures whether predictions are correct.

Despite its superior accuracy, the width model still misclassified some visually clear examples, suggesting that the main limitation lies in the model architecture rather than the optimization procedure. Overall, these results indicate that further improvements would require architectures better suited for image data, such as convolutional neural networks, as well as techniques like data augmentation or transfer learning.